Responda com atenção. Nas questões dissertativas e práticas, justifique suas respostas.

Parte A — Múltipla escolha

1. A propriedade display: flex é aplicada em qual elemento?

   (a) Em cada filho que deve se alinhar.

   (b) No contêiner (pai) cujos filhos se tornarão itens flex.

   (c) No <body>, obrigatoriamente.

   (d) Em qualquer elemento inline.

2. Qual é o valor **padrão** de flex-direction?

   (a) column

   (b) row-reverse

   (c) row

   (d) wrap

3. Com flex-direction: row (padrão), o **eixo principal** é:

   (a) Vertical, de cima para baixo.

   (b) Horizontal, da esquerda para a direita.

   (c) Horizontal, da direita para a esquerda.

   (d) Diagonal, do canto superior esquerdo ao inferior direito.

4. Com flex-direction: column, justify-content distribui os itens: (a) No

   eixo horizontal.

   (b) No eixo vertical (eixo principal).

   (c) Sempre na viewport, ignorando o contêiner.

   (d) Apenas se houver flex-wrap: wrap.

5. A propriedade que alinha os itens ao longo do **eixo cruzado** é:

   (a) justify-content

   (b) flex-direction

(c) align-items

(d) gap

6. O valor padrão de align-items é:

(a) center

(b) flex-start

(c) stretch

(d) baseline

7. justify-content: space-between faz o quê?

(a) Coloca o mesmo espaço *antes*, *entre* e *depois* dos itens.

(b) Coloca o primeiro item no início, o último no fim, e distribui o espaço *somente entre* eles.

(c) Empilha os itens no centro, sem espaço.

(d) Estica cada item até preencher o contêiner.

8. Qual valor de flex-wrap permite que os itens quebrem para a linha seguinte quando não cabem?

(a) nowrap

(b) wrap

(c) row

(d) stretch

9. align-content só tem efeito visível quando:

(a) Há uma única linha de itens.

(b) O contêiner tem flex-wrap: wrap (ou wrap-reverse) **e** mais de uma linha. (c)

Se usa justify-content: center.

(d) Os itens são inline.

10. A diferença prática entre gap e margem nos itens é que gap:

(a) Só funciona em position: absolute.

(b) Define o espaço *entre* os itens, sem criar espaço extra nas bordas externas do con têiner.

(c) Substitui padding do contêiner.

(d) Alinha no eixo cruzado, como align-items.

11. Com flex-direction: row-reverse, os itens:

(a) Empilham de cima para baixo.

(b) Permanecem na ordem do HTML, da esquerda para a direita.

(c) Invertem a ordem visual no eixo horizontal (da direita para a esquerda).

(d) Quebram linha automaticamente.

12. Para centralizar um item **horizontal e verticalmente** em um contêiner flex de altura definida, a combinação correta é:

(a) justify-content: center; e align-items: center;

(b) flex-direction: column; apenas.

(c) flex-wrap: wrap; e gap: 0;

(d) align-content: stretch; apenas.

Parte B — Dissertativas e análise de código

13. Explique, com suas palavras, a diferença entre o **contêiner flex** e o **item flex**. Quem recebe display: flex? O que acontece com os filhos diretos?

14. Considere o código:

```
<div class ="box">
    <div >A </div >
    <div >B </div >
    <div >C </div >
</ div >
```

```
. box {
    display : flex ;
    flex - direction : column ;
    justify - content : center ;
    height : 300 px ;
}
```

Descreva **como** A, B e C ficam dispostos e **por quê**.

15. O que muda no layout da questão anterior se flex-direction: column for **removido** (voltando ao padrão) e o contêiner tiver width: 300px com três filhos largos demais para caber lado a lado? Considere o valor padrão de flex-wrap.

16. Compare justify-content e align-items: (a) sobre qual eixo cada um atua no flex-direction: row;

(b) o que acontece com esses eixos se flex-direction mudar para column.

17. Por que align-content: center **não** centraliza os itens quando o contêiner está com flex-wrap: nowrap e há uma só linha? O que seria necessário para align-content passar a ter efeito?

18. Explique a diferença prática entre usar gap: 16px no contêiner e usar margin-right: 16px em cada item. Cite um problema típico da abordagem com margem.

Parte C — Prática

19. Escreva o CSS para transformar uma lista <ul class="menu» em um **menu horizontal** com Flexbox: itens lado a lado, espaçamento de 12px entre eles (gap) e alinhamento vertical ao centro.

20. Escreva o CSS de um contêiner .galeria que organize vários cards em linhas, **quebrando** quando não couberem, com 16px de espaço entre linhas e entre colunas.

21. Escreva o CSS para **centralizar** um .card no meio da tela (horizontal e vertical). O con têiner é .pagina e deve ocupar pelo menos a altura da viewport (min-height: 100vh).

22. Escreva o CSS de um layout em **coluna**: .pagina empilha header, main e footer de cima para baixo, com 8px de espaço entre as seções. Não use propriedades de item (flex-grow etc.).

23. Escreva o CSS de uma **barra de ferramentas** (.toolbar) com três grupos de botões: o grupo da esquerda no início, o do meio e o da direita nas extremidades, usando justify-content (sem flex-grow).

24. *(Desafio)* Escreva HTML e CSS de uma **faixa de cards** (4 cards) que:

    • usa Flexbox no contêiner;

    • quebra para a linha de baixo em telas estreitas;

    • tem gap de 1rem;

    • na linha, os cards ficam com a mesma altura visual da linha (align-items ade quado);

    • quando houver duas linhas, as linhas fiquem agrupadas no topo do contêiner

    (align-content). Não use flex-grow, flex-shrink, flex-basis, order nem align-self.